Encoding Meta-Characters

The two functions you'll see used the most are `htmlspecialchars()` and `htmlentities()` (and their decoding counterparts). They function in the same way, and the former will encode a smaller set of characters. The following example sums things up:

```
$str = '"&\'<>£¥©®';

print htmlspecialchars($str) . PHP_EOL;
print htmlentities($str);

Output:
&quot;&&#039;&lt;&gt;£¥©®
&quot;&&#039;&lt;&gt;&pound;&yen;&copy;&reg;
```

`htmlspecialchars()` will encode precisely five meta-characters: `" ' & < >` which should be enough to render most markup-containing strings safe for use. To see the full translation table, call `get_html_translation_table(HTML_ENTITIES)`, which will dump an array of characters to entities – call without the `HTML_ENTITIES` constant to see the limited map for `htmlspecialchars()`. Be aware that these functions are aware of the `default_charset` ini setting and if an invalid byte sequence is detected in the input, an empty string is returned. You can specify the encoding of the input with an `encoding:` parameter, for example, `htmlspecialchars($data, encoding: 'cp1252');` would mean the function treats the data as `CP1252` encoded (Windows-1252 Western-European).

Removing Tags

For those hopefully extremely limited cases where markup is *supposed* to be injected (you'll encounter this in ancient, cobbled-together content management systems), `strip_tags()` can be used to remove potentially harmful tags, such as `<script>`. Obviously, a situation like this is less than ideal, and you should vigorously resist implementing such a feature from scratch. Keep your HTML structure in templates, not a database. But this is PHP, some of which has been around for decades; it isn't unheard of to allow people maintaining content to add some structural elements (such as lists or paragraphs) around their text. The problem is that `strip_tags()` isn't very clever and does not pay attention to element attributes. A malicious content editor